How the contract substrate works — intelligence

How the contract substrate works — intelligence

> *Explanatory documentation; rules live in* `DOCTRINE.md`*. Companion to* `docs/HOW-IT-WORKS-architecture.md` *(the machine) and* `docs/HOW-IT-WORKS-economics.md` *(the value layer).*

The substrate's learned layer answers one question — **"for this goal, which tool, used how?"** — from nothing but the ledger. No LLM, no network, no model files at runtime: the weights are compiled into the signed binary, and the training data *is* the append-only contract history. Selection is advisory; the safety gate (`libcontract/src/gate.zig`) is a separate, hard, never-learned authority.

1. Two embeddings, two jobs

Feature-hash embedding — `libcontract/src/embed.zig`. Deterministic hashing-trick vectors (default `DIM = 256`; any dimension via `embedDim`, including the 1536-d RAG-locked dim `contract_embed_1536` uses). Tokens and char-trigrams hash into signed buckets; a fixed stop list approximates IDF. It captures lexical + sub-word overlap (morphology, typos) but knows no synonymy — `embed.zig`'s header says so explicitly ("does not know that door and gate are related"). It remains the dense term for the RAG ranking path in `libcontract/src/ebm.zig` / `libcontract/src/contract.zig`.

Baked BERT — `libcontract/src/bge.zig`. A real learned sentence embedder: bge-small-en-v1.5 as a pure-Zig 12-layer transformer forward pass (`NLAYER = 12`, CLS-pooled, `DIM = 384`) over per-row int8 weights, plus a WordPiece tokenizer — both `@embedFile`'d (`bge_weights.bin`, ~34 MB; `bge_vocab.txt`, 30,522 tokens). Zero runtime dependencies: the same collapse-to-binary doctrine as the rest of the substrate, extended from a lookup table to a genuine transformer.

How it was built and why you can trust it:

- `libcontract/gen-bge-embed.py` (run once at build time) quantizes every matmul weight to per-row int8 + f32 scale and **validates the quantized numpy forward on the routing benchmark before any Zig existed** — per-row quantization measured *zero* accuracy loss vs fp32.
- `bge.zig` then reproduces that numpy reference: its tests assert the Zig tokenizer emits byte-identical HuggingFace token ids and the Zig forward pass matches the golden CLS vectors (`bge_ref.bin`) at cosine > 0.999. The erf approximation (Abramowitz-Stegun 7.1.26) is shared between generator and Zig so the numerics agree.
- Contract of `bge.embed`: always-valid output — empty/unembeddable text yields the zero vector, never NaN.

2. The energy router — the ledger IS the weights

`libcontract/src/route.zig`, one verb: `recommend()` (CLI `contract recommend "<goal>"`, C-ABI `contract_recommend`, aiko-cli FFI `native.recommend`). On every call it re-derives its "weights" from the ledger:

- **Fold**: every `satisfied` contract folds its goal's bge vector into the prototype of the binary its `action` named (reward +1); every `died` row inhibits (–1). The prototype is a reward-weighted running-mean centroid (West 1979) — O(1) per fire, the same `Prototype` mechanism proven at unit level in `libcontract/src/ebm.zig` (Hebbian potentiation, negative rewards as the contrastive phase).
- **Settle**: energy = –cosine(goal, prototype); the recommendation is the argmin-energy binary. A net-negative binary (weight ≤ 0) has infinite energy and is never chosen.
- **Honest cold**: nothing learned -> `null`. Never a fabricated route.

Pure ledger, no lexicon. There is no hardcoded tool list and no cold-start seed: a binary becomes a candidate only after a real fire names it. (A hand-written lexicon was tried and deleted — its broad seeded prototypes *stole* goals from trained binaries, measured 40.7% vs 55.6% without; the fix-then-delete history lives in git.) Discovery of new tools is the agent's job; the router is the exploit half — a cache of what worked.

Self-documented usage. Because aiko-cli records the *full command* as the action (not just the binary name), `recommend()` returns both the selected binary and the nearest **proven command** — the satisfied row of that

binary whose goal sits closest (by bge cosine) to the new goal — as JSON `{binary, example, score}`. A novel goal gets not just "use grep" but "a similar goal was satisfied by `grep -rn "pattern" src/`": a self-written skill grounded in what actually worked in this environment, retrieved by meaning.

3. The closed loop through aiko-cli

The router learns from real work, automatically (aiko-cli repo):

- **Capture** — `src/contract/tool-neurons.ts`: every gated command tool that *succeeds* records its full command line (`commandLine`, capped); the turn drains them via `takeTurnCommands()`.
- **Learn** — `src/contract/agency.ts learnRoute(goal, command)`: each drained command becomes a satisfied contract (plan = the user's turn task, action = the exact command) — one ledger row, which is all the router needs.
- **Advise** — `src/agent/agent.ts`: at turn start, `native.recommend()` injects a `LEARNED HINT` into the system prompt — the tool plus its proven exemplar, explicitly marked "a learned prior from past runs, not a rule". The model remains free to ignore it; the gate still checks whatever actually runs.

Fires that satisfy sharpen the instinct; fires that die inhibit it. The loop is best-effort by construction — a missing dylib turns the whole layer off without breaking a turn.

4. The honest benchmark

`libcontract/src/route_bench.zig` (run: `zig test libcontract/src/route_bench.zig -OReleaseFast`) uses the *shipped* embedding + energy model on 54 natural-language goals × 6 binaries, leave-one-out (train on 53, test the held-out goal — generalization by distance, not memory). Current shipped numbers:

metric	value
random / keyword-in-goal	16.7% / 1.9%
learned centroid (pure ledger, LOO)	**88.9%**
per-binary	git 78 · docker 89 · grep 89 · curl 100 · npm 89 · tar 89
learning curve	k=1 85.4% · k=2 88.1% · k=3 94.4%
1-NN comparison (same embedding)	81.5% — the centroid wins

The test's witnesses assert only what the data supports: > 3× random and ≥ 85% (see the `expect` lines at the bottom of `route_bench.zig`).

The improvement arc, kept honest (same harness throughout): the feature-hash embedding capped routing at 44%; IDF-approx stopwords lifted it to 57% but grep (pure synonymy) sat at 11%. Candidate embedders were then measured on an identical Python replica of the harness (validated by reproducing the Zig number exactly): GloVe-100 66.7%, nomic-embed-text-v1.5 87.0%, **bge-small 88.9%**, Qwen3-Embedding-0.6B 96.3%. Qwen is the known ceiling — 20× bge's size for +7 points, rejected as a bundle; bge-small won on accuracy-per-MB and was ported to pure Zig with no further loss.

5. Limits — stated, not hidden

- **Advisory only.** ~9/10 right is excellent for a prior and unacceptable for enforcement; that is why safety stays in the never-learned gate.
- **O(N) settle.** `recommend()` scans every prototype per call — fine at tens of tools, needs an ANN index at thousands.
- **No zero-shot.** Pure-ledger means a never-used tool is invisible until its first satisfied use (one LLM roundtrip of "exploration cost"). That is a chosen trade: no fabricated routes from typed descriptions.
- **Benchmark ≠ production.** 88.9% is on a curated 6-way task; the number to watch is performance on the real ledger as it grows.

Further reading

- The machine underneath: `docs/HOW-IT-WORKS-architecture.md`
- The value layer: `docs/HOW-IT-WORKS-economics.md`

- Phase plan + status for this layer: `docs/PLAN-contracts-as-onchain-ebm-weights.md`